ResTrack - Ultimate Codebase Guide & Code Review

1. System Overview

ResTrack is a full-stack Research Tracking web application designed to manage the lifecycle of research studies. The system enables researchers to submit studies, reviewers to evaluate them, TRB chairs to provide final oversight, and administrators to manage users and monitor the overall workflow.

Architecture

- **Frontend**: React.js with modular component-based styling (CSS Modules)
- **Backend**: Node.js and Express.js REST API
- **Database**: PostgreSQL (Cloud-hosted via Neon)
- **Deployment**: Vercel (Frontend), Render (Backend)

---

2. Backend Architecture (`restrack-backend`)

The backend follows a typical Model-View-Controller (MVC-lite) pattern using Express routing.

2.1 Core Infrastructure

- `server.js`: The main entry point initializing the Express app, CORS, JSON body parsing, and routing.
- `db.js`: Contains the PostgreSQL connection pool setup to communicate with the Neon cloud database.
- `.env`: Environment variables (e.g., `PORT`, `DB_URL`, `JWT_SECRET`).

2.2 Routes (`/routes`)

- `auth.js`: Handles user authentication, login validation, and JWT generation.
- `users.js`: Handles user management operations (CRUD operations, soft deletes, role updates) restricted to Admin users.
- `studies.js`: The core business logic for the research pipeline. Handles submitting new studies, updating study statuses, assigning reviewers, processing evaluations, and managing attachments. Researcher-facing list/detail/update paths scope data by creator, corresponding author, and `research_authors` (see section 2.5).
- `dashboard.js`: Aggregates key system statistics and analytics data for the Admin dashboard.
- `notifications.js`: Handles system notifications and status updates for users.

2.3 Middleware (`/middleware`)

- `requireAuth.js`: Validates JWT tokens provided in the request headers to protect authenticated routes and endpoints.

2.4 Database Scripts (`/scripts`, `/migrations`)

- Includes utility scripts for database seeding (`seed.js`) and altering schema (`alter_db.js`), allowing for easy setup of initial admin users and necessary database tables.

2.5 Researcher study visibility (how users only see “their” studies)

Visibility is enforced on the **server**, not by trusting the React UI. Every protected studies call uses `requireAuth`, which decodes the JWT and sets `req.user.id` (and role when present). The database then filters by that user id.

Listing studies (`GET /api/studies/my` in `routes/studies.js`)

- The query returns a study only if the current user is linked to it in at least one of these ways: `research_studies.created_by`, `research_studies.corresponding_author_id`, or a row in `research_authors` for that study and user.
- That means researchers see studies they **created**, act as **corresponding author** for, or are listed as **co-authors** on—not arbitrary submissions from other labs.
- **Admins** are handled in the same handler: if the resolved role is `Admin`, the `WHERE` clause effectively broadens so they can list all studies (same endpoint, role-aware SQL).

Opening or editing one study

- `GET /api/studies/:id` runs an access check before returning the payload. For a **Researcher** (and similarly for creator/co-author), access requires the same creator / corresponding author / `research_authors` relationship. If the user has no match, the API responds with **404** (“Study not found”), which avoids leaking whether an id exists to unauthorized users. TRB, Reviewer, and Admin paths add their own role-specific rules (assignments, status, etc.).
- `PUT /api/studies/:id` (researcher updates) uses the same ownership/co-author predicate; there is no “admin bypass” on this route—admins use `PUT /api/studies/:id/admin-update` instead.

Dashboard counts

- `GET /api/dashboard/researcher/stats` (`routes/dashboard.js`) aggregates counts from a CTE that applies the same three-way link (`created_by`, `corresponding_author_id`, or `research_authors`) so dashboard numbers cannot include other users’ studies.

Frontend alignment

- Researcher-facing lists (for example `Studies.js` and `RecentStudies.js`) call `/api/studies/my` with the Bearer token rather than a generic “all studies” endpoint, so the browser only ever receives rows the backend has already scoped to that user (plus the co-author and corresponding-author cases above).

---

3. Frontend Architecture (`restrack-frontend`)

The frontend is a single-page React application structured logically by component functionality.

3.1 Application Entry

- `index.js` / `App.js`: Bootstraps the React application, sets up React Router (`react-router-dom`) for navigation, and handles initial app context and global routing state based on authentication.
- `config.js`: Maintains environment-specific constants, like the backend API base URL (`REACT_APP_API_URL`).

3.2 Layout (`/src/components/layout`)

- `Layout.js` & `Layout.css`: A reusable layout wrapper that provides the navigation sidebar, header, and user profile summary. It acts as the shell for all authenticated pages and dynamically adjusts navigation links based on the user's role (Admin, Researcher, Reviewer, TRB).

3.3 Pages (`/src/components/pages`)

The application pages are separated logically by role and feature:

Authentication

- `Login.js`: User login interface handling credential submission and JWT storage.
- `Signup.js`: Interface for new user registration.

Admin Role

- `DashboardAdmin.js`: Displays system-wide analytics.
- `ManageUsers.js`: Interface for Admins to view, edit, change roles, and soft-delete users.
- `ReviewQueue.js` & `ReviewQueueStudy.js`: Provides an overarching view of all studies in the pipeline and their progression.

Researcher Role

- `DashboardResearcher.js`: Researcher landing page.
- `Studies.js`: Lists all studies submitted by the researcher.
- `NewStudy.js`: A comprehensive form for researchers to submit a new research study.
- `EditStudy.js` / `SpecificStudy.js`: Views to modify or review the details of their specific submissions.

Reviewer Role

- `DashboardReviewer.js`: Reviewer landing page.
- `Assignments.js`: Lists studies assigned to the reviewer.
- `AssignedStudy.js`: Interface for the reviewer to view attachments, provide feedback, and submit their approval/rejection.

TRB Chair Role

- `DashboardTRB.js`, `AssignmentsTRB.js`, `AssignedStudyTRB.js`: Dedicated views for the TRB Chair to provide higher-level oversight and final approvals on studies that passed initial review.

---

4. Code Review & Best Practices Assessment

4.1 Strengths

- **Role-Based Access Control (RBAC):** Excellent separation of concerns across different user roles both in frontend routing (`Layout.js` conditional rendering) and backend data serving.
- **Modular Styling:** The use of CSS Modules (e.g., `SpecificStudy.module.css`) correctly scopes styles to components, preventing CSS collisions and keeping the codebase maintainable.
- **RESTful API Design:** Backend routes are intuitively structured, mapping directly to system resources (`/studies`, `/users`).

4.2 Areas for Improvement (Constructive Feedback)

- **Centralized Error Handling:** Consider implementing a global error handling middleware in `server.js` for the Express backend rather than repetitive `try-catch` blocks sending `500` status codes across all routes.
- **Form Validation:** Ensure comprehensive frontend validation using libraries like `Formik` and `Yup` in forms like `NewStudy.js` and `ManageUsers.js`, backed by robust server-side validation using `express-validator` to prevent malformed data injection.
- **Pagination:** As the system scales, endpoints like `GET /users` or `GET /studies` should implement pagination to limit payload size and improve frontend render times.
- **Security:** Ensure that file uploads (attachments) are strictly sanitized to prevent malicious execution, and implement rate limiting on the `/auth` routes to prevent brute force attacks.

---

5. Deployment Overview

- **Frontend:** Hosted on Vercel (`vercel.json` present), allowing for seamless CI/CD integration with GitHub.
- **Backend:** Hosted on Render, providing a stable Node.js runtime.
- **Database:** Neon PostgreSQL ensures serverless, scalable relational data management.
- **Environment Management:** Critical keys (`DB_URL`, `JWT_SECRET`) are correctly excluded from version control (`.gitignore`) and tracked via `.env.example`.